

PROGRAMMIERUNG I

Organisatorisches und Einstieg

KONTAKT

phillip.goellner.dhbw@protonmail.com

<https://www.pgoellner.com/#!/lecture>



Curriculum

Kapitel 0

Programmieren und Programmiersprachen
Klassifikation von Programmiersprachen
Übersicht zu und Installation von Java
Zusätzliche Lernmittel

Kapitel 1

Einordnung und Hintergründe zu Java
Versionskontrolle mit Git
Anweisungs-Sequenzen
Verzweigung/Konditionen, Boolesche Ausdrücke
Schleifen/Iteration
Variablen, Konstanten, Datentypen, Casting
Pair Programming

Kapitel 2

Methoden, Main-Methode
Software-Testing
Objektorientierung, Klassen, Objekte
Kapselung, Akzessor-Methoden
Klassendiagramm, Vererbung
Method-Overloading, Method-Overriding
Instanz-Variablen/Methoden, Statische Variablen/Methoden

Kapitel 3

Array-Datentypen
Polymorphie, Interfaces, Abstrakte Klassen
Assoziationen
Objekthierarchien, Rekursion
Packages, Visibility
Imports
Fehlerbehandlung

Kapitel 4

Erweiterte Dokumentation mit JavaDoc
Code-Qualität und fortgeschrittenes Testen
JVM Stack, Heap
Refactoring

Begriffsdefinitionen

Programmierung (Coding)

Die Tätigkeit, ein Computerprogramm zu erstellen, d.h. Umsetzung eines Softwareentwurfs in Quellcode.

Alternativ: Die Tätigkeit, einem dummen, aber außerordentlich fleißigen Computer zu erklären, was dieser zu tun hat – mit Hilfe einer Programmiersprache.

Programmiersprache

Eine formale, logische und menschenverständliche Sprache, mit der Programmabläufe definiert und letztlich von einem Computer ausgeführt werden

Merke: Der Prozessor kann nur Maschinencode (d.h. Folgen von Bits) ausführen.
Die menschenverständliche Programmiersprache dient also hauptsächlich dazu, dem Menschen ein effektiveres und effizienteres Programmieren zu ermöglichen

Quellcode/Quelltext

Der für den Menschen lesbare, in einer Programmiersprache geschriebene Text eines Computerprogramms; Der Quelltext eines Programms kann auf mehrere Teile/Dateien verteilt sein.

Klassifikation Von Programmiersprachen

~

Programmiersprachen unterscheiden sich teils stark voneinander.

Man kann sie anhand verschiedener Dimensionen bzw.

Paradigmen klassifizieren und unterscheiden, zum Beispiel:

- [Low-Level](#) (z.B. Assembly) vs. [High-Level](#) (z.B. Java) vs. [Very-High-Level](#) (siehe DSL)
- [General-Purpose-Lang.](#) (z.B. Java, C) vs. [DSL/Domain-Specific-Lang.](#) (z.B. SQL)
- [Imperative](#) (die meisten) vs. [Declarative](#) (z.B. SQL, Prolog, Lisp, Haskell)
- [Interpreted](#) (z.B. Bash) vs. [Compiled](#) (z.B. Java) vs. [Assembly](#) (Prozessor-spezifisch)
- [Statically Typed](#) (z.B. Java) vs. [Dynamically Typed](#) (z.B. JavaScript)
- [Object-Oriented](#) (die meisten modernen Sprachen) vs. [Structured](#) (z.B. ALGOL, PL/1)
- [Light-Core](#) (z.B. C, Ruby) vs. [Large-Core](#) (z.B. Java, VB .NET)
- [Text-Based](#) (die meisten) vs. [Visual](#) (z.B. Lego NXT-G, WebML, Simulink)

Darüber hinaus kann danach unterschieden werden, wie häufig und gravierend die Programmiersprache erweitert bzw. angepasst wird:

- keine bzw. vernachlässigbare Weiterentwicklung (z.B. ALGOL, FORTRAN)
- evolutionäre, gemäßigte Weiterentwicklung (z.B. Java, C++, Ruby, Python)
- rapide und gravierende Veränderung (z.B. Groovy, Go, Dart, Scala)

Java

~

Einordnung

Die Programmiersprache Java lässt sich folgendermaßen in das eben vorgestellte Schema einordnen:

- **High-Level** (entfernt an menschliche Sprache angelehnt)
- **General-Purpose-Language** (vielseitig verwendbar)
- **Imperative** (Definition der Abfolge von Anweisungen)
- **Compiled** (vor der Ausführung muss der Quellcode übersetzt werden)
- **Statically Typed** (Variablen können nur definierte Datentypen speichern)
- **Object-Oriented** (Definition, Erzeugung und Interaktion von Objekten)
- **Large-Core** (JDK 11 ist installiert ca. 300MB groß)
- **Text-Based** (der Quellcode liegt als Textdatei vor: *.java)
- **Evolutionäre, gemäßigte Weiterentwicklung** (mit Rückwärtskompatibilität)



Kurzübersicht der Historie:

(nicht klausurrelevant)

- 1991: James Gosling, Mike Sheridan und Patrick Naughton beginnen bei Sun Microsystems mit der Arbeit am Java-Projekt (nach der Kaffeesorte benannt)
- 1995: Sun Microsystems veröffentlicht Java 1.0
- 1998: mit Java 1.2 werden die Enterprise Edition und Micro Edition eingeführt
- 2006: Sun Microsystems stellt Java (Java 5) unter eine Open-Source Lizenz (GPL)
- 2009: Oracle übernimmt Sun Microsystems und damit Java
- 2014: Release von Java 8 (Long-Term-Support)
- 2018: Release von Java 11 (Long-Term-Support)
- 2019: Oracle ändert Java Release- und Support-Modell. Aufstieg von [OpenJDK](#), [AdoptOpenJDK](#), [Zulu](#)



Java: Installation Und Entwicklung

~

Wer lediglich Java-Programme ausführen möchte, benötigt:

Java SE Client Runtime Environment (JRE)

[Download](#) (ca. 30MB) – installiert ca. 150MB

Wer Java-Programme entwickeln möchte, benötigt:

Java Software Development Kit (JDK)

[Download](#) (ca. 150 MB) – installiert ca. 300 MB
beinhaltet auch eine vollwertige JRE

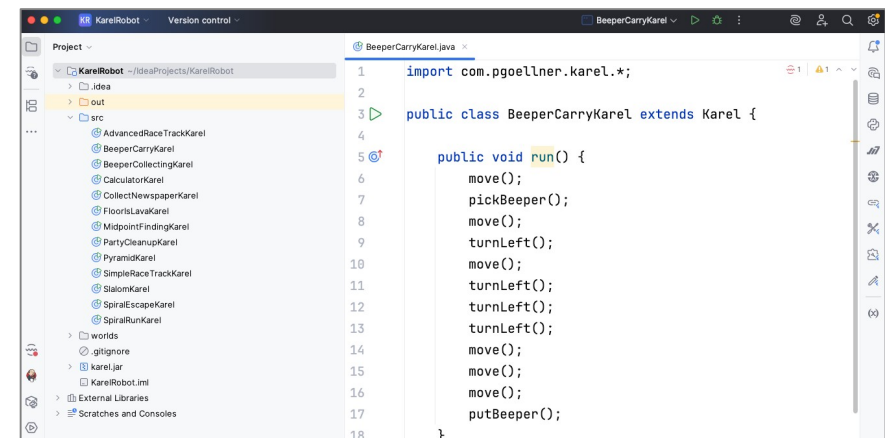
Sowie entweder: Text-Editor

Editor (z.B. [Notepad++](#), [Atom](#)) zum Schreiben des Quelltexts
Konsole zum Aufrufen des Compilers und zum Ausführen des Bytecodes
Dieser Ansatz ist sehr mühsam und ab einer gewissen Komplexität nicht mehr praktikabel.

Oder: Entwicklungsumgebung

Integrated Development Environment (IDE) z.B. [IntelliJ IDEA](#) (ca. 1 GB)
oder [Eclipse IDE](#) (ca. 250MB)

Durch die sehr große Anzahl an Funktionen steigern IDEs einerseits die Programmier-Effizienz, erfordern andererseits aber eine lange Einlernphase.



IntelliJ IDE

Zusätzliche Lernmittel (Auswahl)

X

Literatur

[Java von Kopf bis Fuß](#) – Lehrbuch für Java-Einsteigende

[Java ist auch eine Insel](#) – kostenloses E-Book für Java-Einsteigende

[Effective Java: A Programming Language Guide](#) – Tipps zur Verbesserung des Programmierstils für fortgeschrittene Java-Entwickelnde

Online-Kurse

(Zusätzlich zum Literaturstudium oder statt diesem)

[Treehouse](#) – große Anzahl von Online-Kursen zu Java und vielen anderen Themen

[Codeschool](#) / [Pluralsight](#) – exzellent gemachte Online-Kurse, primär zu Webentwicklungs-Themen

Hilfe bei der Entwicklung

(Auch fortgeschrittene Entwickelnde kennen nicht alle Lösungen)

Die Suchmaschine des Vertrauens (z.B. [Google](#), [Bing](#), [DuckDuckGo](#))

LLMs wie ChatGPT, Copilot oder Claude (wobei die Qualität der Antworten sehr durchwachsen sein kann)

[Stack Overflow](#) – ein allgemeines Entwicklungsforum zu Programmier-Problemen mit Community-Antworten (hilfreichste Antwort zuerst)

[Java API Documentation](#) – Offizielle Dokumentation aller Core Java-APIs

[Java-Forum](#) – deutsche Java-Entwicklungs-Community mit Frage-Antwort-Forum

Legende Zum Skript

Das bedeutet der Inhalt der Seite ist nicht klausurrelevant

X

Beschreibung	Beispiele
Java-Keywords	<code>void, int</code>
Hervorgehobene oder gekennzeichnete Komponenten/Namen	<code>true, Condition, &&, MyClass</code>
Normaler Quellcode (vor allem gekennzeichnet durch die Schriftart)	<code>move();</code>
Links	Beispiel-Link
Wichtige Begriffe	Klasse, Methode

Das signalisiert, dass ein grundlegendes Verständnis ausreicht

Syntax-Kasten:

```
{  
  // Hier steht dann Java Code!  
}
```

Hierin stehen ergänzende Erklärungen



Photo by [Zoë Reeve](#) on [Unsplash](#)

PROGRAMMIERUNG I

Kapitel 1: Einführung in Java Grundlagen

Wie Java Funktioniert

Damit Computerprozessoren ein Programm ausführen können, muss dieses als Maschinencode vorliegen - eine prozessorarchitektur-spezifische Folge von Bits.

Bei Java erfolgt die Übersetzung vom menschenlesbaren Quelltext in den Computer-verständlichen Maschinencode in zwei Schritten:

Kompilation mit dem Java Software Development Kit (JDK)

Java-Compiler (javac)

- übersetzt Java-Quellcode (`*.java` Textdateien) in Java-Bytecode (`*.class` Binärdateien)
- prüft den Quellcode auf syntaktische (formale) Fehler

Java-Bytecode

- ist noch „high-level“ (also prozessorarchitektur-unabhängig)
- kann direkt von allen Geräten mit installiertem Java ausgeführt werden, d.h. z.B. zum Download angebotene Java-Programme liegen im Bytecode vor
- ist nur noch schwer lesbar, kann aber häufig wieder in Quellcode zurück-übersetzt werden
- kann nicht nur aus Java-Quellcode erzeugt werden. Mittels anderer Compiler kann auch aus Python, Ruby oder PHP-Quellcode lauffähiger Java-Bytecode erzeugt werden.

Ausführung in der Java Runtime Environment (JRE)

Java Virtual Machine (JVM)

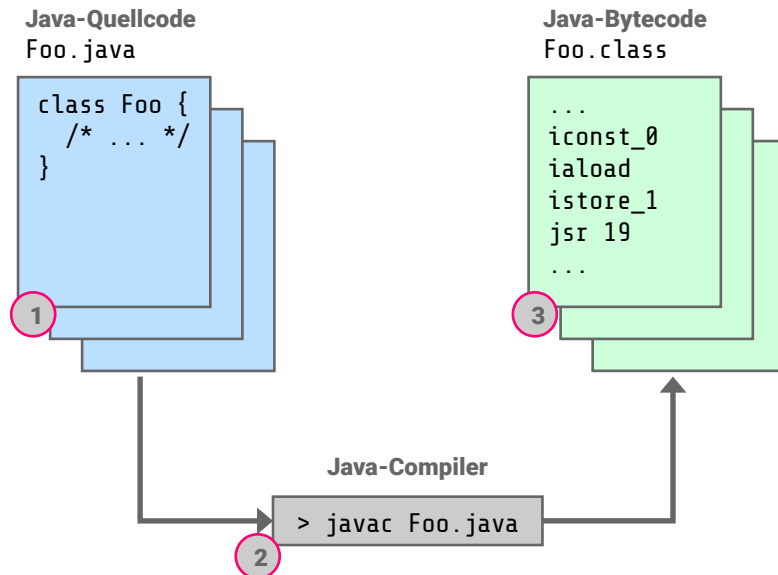
- übersetzt Java-Bytecode (`*.class`) in prozessor-spezifischen Maschinencode
- verwendet seit Version 1.3 einen Just-in-Time (JIT) Compiler (HotSpot), welcher den Bytecode bedarfsweise in Blöcken optimiert und dann in Maschinencode umwandelt

Java Class Library (JCL)

ist ein Paket (`rt.jar`) aus Java-Bytecode-Dateien, welche die am häufigsten verwendeten Grundfunktionalitäten bereitstellen (z.B. Dateizugriff, grafische Oberflächen) ist Teil der JRE und damit auf jedem Gerät verfügbar, auf welchem Java installiert ist

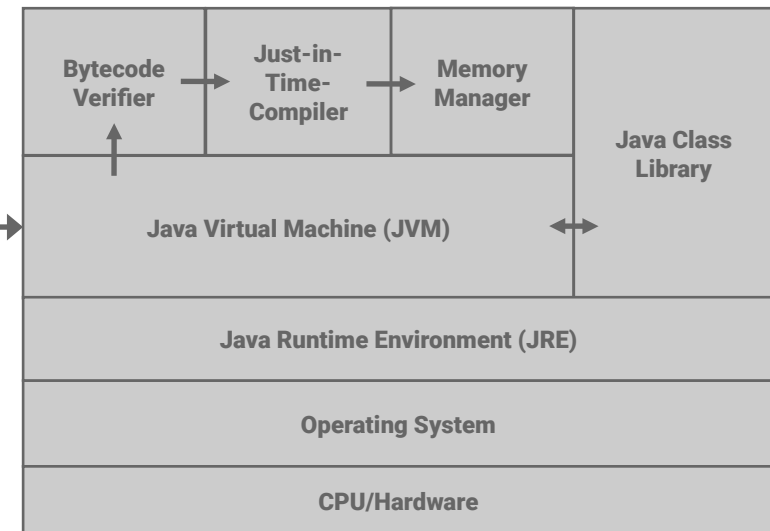
Kompilation

mit dem Java Software Development Kit (JDK)



Ausführung

in der Java Runtime Environment (JRE)



Karel J. Robot

Karel J. Robot ist eine Simulationsumgebung zum Lernen und Üben der Programmierung. Ursprünglich entwickelt 1981 von Richard E. Pattis (Stanford University)

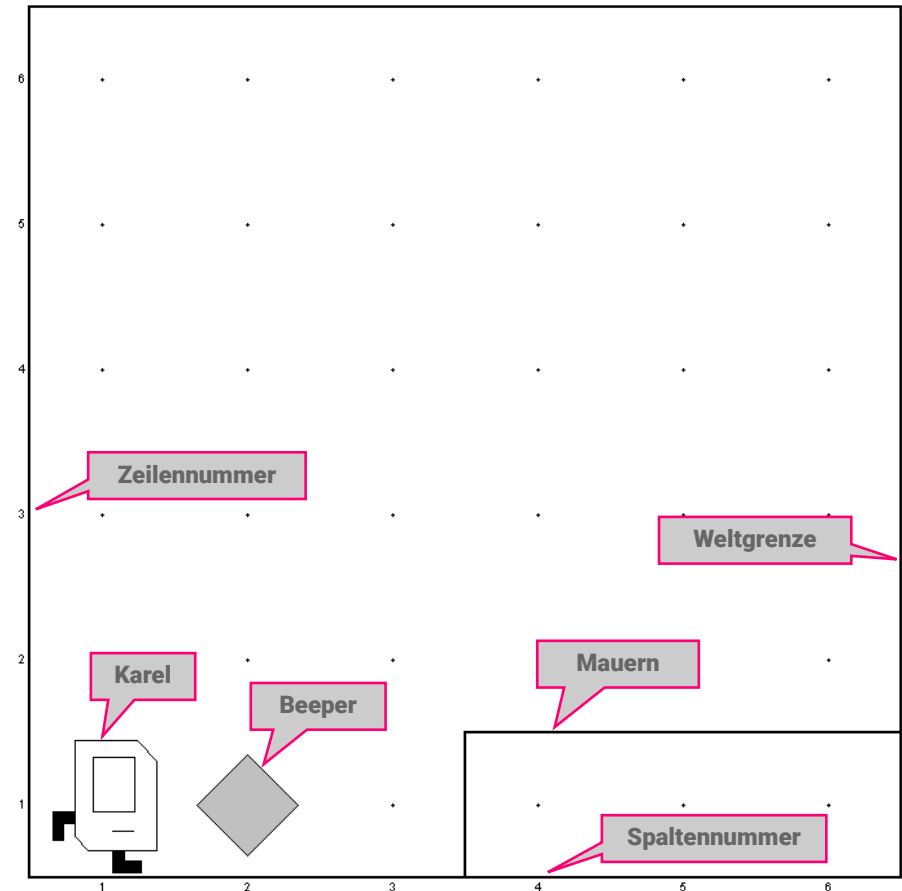
Karels Welt:

- Gitternetz-Welt mit nummerierten Zeilen und Spalten
- Karel kann sich im Gitternetz bewegen.
- Beeper können von Karel aufgenommen oder abgelegt werden.
- Karel kann nicht durch Mauern hindurchgehen.

Karels Grundbefehle (**Methoden**):

- `move()` → laufe einen Schritt nach vorne
- `turnLeft()` → 90° Drehung nach links
- `pickBeeper()` → hebe Beeper auf
- `putBeeper()` → lege Beeper ab

Karel hat außerdem noch einige Sensoren, welche ihm Informationen über seine Umgebung vermitteln (dazu später mehr)



Hinweise zum Quellcode

Der Quellcode aus Aufgabe 1 der Übungen wird hier anhand seiner Komponenten erklärt.

```
1 import com.pgoellner.karel.*;
2
3 public class BeeperCarryKarel extends Karel {
4
5     public void run() {
6         move();
7         pickBeeper();
8         move();
9         turnLeft();
10        move();
11        turnLeft();
12        turnLeft();
13        turnLeft();
14        move();
15        move();
16        move();
17        putBeeper();
18    }
19
20 }
```

1. Zunächst ist nur der magenta umrahmte Bereich interessant. Das Drumherum wird vorerst als gegeben hingenommen – es wird später noch genau erklärt.

2. Die verschiedenen Anweisungen werden **sequenziell** abgearbeitet – Zeile für Zeile, von oben nach unten.

3. Die einzelnen Anweisungen sind **Methodenaufrufe**, daher enden sie mit (). Dazu später mehr.

4. In Java muss jede individuelle Anweisung mit einem **Semikolon** abgeschlossen werden.

5. Zu jeder öffnenden Klammer gehört auch eine schließende Klammer. In Java haben runde Klammern (*parentheses*), geschweifte Klammern *{curly braces}* und eckige Klammern *[square brackets]* jeweils komplett unterschiedliche Bedeutungen.

Kommentieren

Um anderen Personen (und sich selbst) das Verstehen des geschriebenen Quellcodes zu erleichtern, kann man geschriebenen Code mit **Kommentaren** versehen.

Kommentare sind allein an den Menschen gerichtet. Sie werden bei der Kompilation entfernt und bei der Ausführung des Programms ignoriert.

Syntax

One-Line:

```
// Wiederholt alles in einer Endlosschleife  
while (true) {
```

Multi-Line:

```
/* Goal: Pick up all beepers on the red fields and place all picked up  
   beepers on the blue field  
*/  
public class CalculatorKarel extends Karel {
```

JavaDoc:

```
/**  
 * Erzeugt das heutige Datum in folgendem Format: dd.MM.yyyy  
 *  
 * @return formatierten DatumString  
 */  
public static String getHeuteDDMMYYYY() { 2 usages
```

Zu **JavaDoc** gibt es später mehr

Konventionen

- Ein Kommentar vor jeder Klasse, der beschreibt wozu die Klasse dient
- Ein Kommentar vor jeder Methode, der den Zweck, sowie die Ein- und Ausgaben beschreibt
- **Vorsicht:** Konventionen müssen auch hinterfragt werden! Veraltete oder falsche Kommentare können schnell mehr Schaden anrichten als sie Gutes tun. Es lohnt sich häufig besser lesbaren Code zu schreiben statt ihn zu kommentieren

Grundlegende Komponenten

Um komplexere Probleme lösen zu können, wird mehr benötigt als nur eine Aneinanderreihung von Methodenaufrufen. Nahezu alle imperativen Programmiersprachen unterstützen deshalb zusätzlich die folgenden Kernelemente, auf die der Reihe nach eingegangen wird:

Kernelement	Java-Keywords (Auswahl)	Pseudocode-Beispiele
Schleifen	<code>for, while, do, break, continue</code>	▪ SOLANGE der Weg frei ist GEHE gerade aus ▪ FÜR ALLE Zahlen von 1 bis 100 PRÜFE ob Primzahl ▪ SOLANGE hier noch Beeper liegen HEBE einen Beeper auf
Bedingungen/ Verzweigungen	<code>if, else, switch</code>	▪ WENN der Weg geradeaus versperrt ist DANN prüfe ob der Weg links oder rechts frei ist ▪ WENN die gesuchte Datei nicht gefunden wurde DANN zeige eine Fehlermeldung an
Variablen/ Konstanten	<code>final</code>	▪ MERKE die aktuelle Zeilennummer ▪ ERHÖHE die Anzahl der Beeper um 1
Parametrisierte Methoden	<code>return, void</code>	▪ Hebe alle Beeper in der aktuellen Reihe auf ▪ Finde alle Dateien auf der Festplatte, die den angegebenen Namen haben

Die beiden erstgenannten Kernelemente (**Schleifen** und **Verzweigungen**) werden auch **Kontrollstrukturen** genannt. Mit ihnen kann der Programmfluss (d.h. der Ablauf des Programms) gesteuert (kontrolliert) werden.

Boolesche Ausdrücke

Die Entscheidung, ob z.B. der **if**-Teil einer Verzweigung abgearbeitet werden soll oder nicht hängt vom Ergebniswert des entsprechenden **Booleschen Ausdrucks** ab.

Boolesche Ausdrücke ergeben immer einen der beiden **Booleschen Werte** **true** oder **false**.

Mathematische Vergleichsoperationen liefern ebenfalls Boolesche Werte zurück, z.B.:

<code>if (3 == 3)</code>	<code>→ true</code>	(gleich)
<code>if (1 < 2)</code>	<code>→ true</code>	(kleiner als)
<code>if (1 > 2)</code>	<code>→ false</code>	(größer als)
<code>if (2 <= 2)</code>	<code>→ true</code>	(kleiner oder gleich)
<code>if (1 != 1)</code>	<code>→ false</code>	(ungleich)

Boolesche Werte bzw. Ausdrücke können miteinander zu komplexen Booleschen Ausdrücken verknüpft werden. Java unterstützt eine [Vielzahl](#) von **Verknüpfungs-Operatoren**. Hier die wichtigsten:

Typ	Operator-Syntax	Bedeutung	Beispiele
Und	<code>Condition1 && Condition2</code>	WENN Cond1 UND Cond2 (beide müssen erfüllt sein)	<code>if (beeperIsPresent() && frontIsClear())</code>
Oder	<code>Condition1 Condition2</code>	WENN Cond1 ODER Cond2 (mindestens eine muss erfüllt sein)	<code>if (beeperIsPresent() frontIsClear())</code>
XOR	<code>Condition1 ^ Condition2</code>	WENN ENTWEDER Cond1 ODER Cond2 (exakt eine muss erfüllt sein)	<code>if (leftIsClear() ^ rightIsClear())</code>
Negation	<code>! Condition</code>	WENN NICHT Condition	<code>if (!beeperIsPresent())</code>
Klammern	<code>Condition1 && (Condition2 Condition3)</code>	WENN Condition1 UND (Condition2 ODER Condition3)	<code>if (frontIsClear() && (leftIsBlocked() rightIsBlocked()))</code>

Auch verschiedene **Methoden** von Karel liefern Boolesche Werte zurück, z.B.:

```
frontIsClear()
beepersPresent()
```

Schleifen

Schleifen (Loops) dienen der mehrfachen Wiederholung eines bestimmten Programmteils, solange eine bestimmte Bedingung erfüllt ist:

„SOLANGE x TUE y“

Die WHILE-Schleife

Condition: Boolescher Ausdruck, der die Anzahl der Wiederholungen begrenzt (Bedingung)

LoopBody: Code der zu wiederholenden Anweisungen

```
Syntax:  
while (Condition) {  
    LoopBody  
}
```

```
while(frontIsClear()) {  
    move();  
}
```

Bei der **while**-Schleife befindet sich der **Schleifenkopf** (die **Condition**) vor dem **Schleifenrumpf** (**LoopBody**). Ist die **Condition** gleich **false**, wird der **LoopBody** kein einziges Mal ausgeführt.

Die DO-Schleife

```
Syntax:  
do {  
    LoopBody  
} while (Condition);
```

```
do {  
    move();  
} while (frontIsClear());
```

Bei der **do**-Schleife befindet sich der **Schleifenkopf** nach dem **Schleifenrumpf**. Dadurch wird der Schleifenrumpf immer mindestens einmal ausgeführt, selbst wenn die **Condition** gleich **false** ist.

Die FOR-Schleife

- Counter:** Wert, der sich mit jeder Wiederholung ändert (z.B. Ganzzahl: `i`)
- Condition:** Boolescher Ausdruck, der die Anzahl der Wiederholungen begrenzt (z.B. Bedingung: `i<=4`)
- Increment:** Ausdruck, der den **Counter** bei jeder Wiederholung ändert (z.B. um 1 erhöht mittels: `i++`)
- LoopBody:** Code der zu wiederholenden Anweisungen

Syntax:

```
for (Counter; Condition; Increment) {  
    LoopBody  
}
```

Innerhalb des **LoopBody** kann mit den folgenden Keywords der Ablauf der Wiederholungen beeinflusst werden:

- break;** Breche die Ausführung der gesamten Schleife ab und fahre nach der Schleife fort.
- continue;** Breche die Ausführung der aktuellen Wiederholung ab und mache mit der nächsten Wiederholung weiter.

```
for (int i = 1; i <= 4; i++) {  
    System.out.println("Number: " + i);  
}
```

```
for (int i = 1; i <= 3; i++) {  
    turnLeft();  
}
```

Verzweigungen

Mit Bedingungen können in Abhängigkeit des Systemzustands unterschiedliche Anweisungen ausgeführt werden:

„WENN x DANN y“ bzw. „WENN x DANN y ANSONSTEN z“

Die IF-Verzweigung

Condition: Boolescher Ausdruck, der die Wahrheitswerte `true` oder `false` annehmen kann

IfBody: auszuführende Anweisung(en) falls **Condition1** `true` ist

ElseIfBody: auszuführende Anweisung(en) falls alle vorherigen Conditions `false` sind, aber **Condition2** `true` ist (*optional und mehrfach möglich*)

ElseBody: auszuführende Anweisung(en) falls alle vorherigen Conditions `false` sind (*optional*)

Syntax (einfaches IF):

```
if (Condition1) {  
    IfBody  
}
```

```
if (beeperIsPresent()) {  
    pickBeeper();  
}
```

Syntax (IF ELSE):

```
if (Condition1) {  
    IfBody  
} else {  
    ElseBody  
}
```

```
if (frontIsClear()) {  
    move();  
} else {  
    turnLeft();  
}
```

Syntax (IF ELSE-IF ELSE):

```
if (Condition1) {  
    IfBody  
} else if (Condition2) {  
    ElseIfBody  
} else {  
    ElseBody  
}
```

```
if (frontIsClear()) {  
    move();  
} else if (leftIsClear()) {  
    turnLeft();  
} else {  
    turnLeft();  
    turnLeft();  
    turnLeft();  
}
```

Die SWITCH-Verzweigung

- Variable:** Wert (z.B. Zahl), auf den geprüft wird
- Option:** Wert einer von mehreren Optionen
- CaseBody:** Wenn **Variable** gleich **Option1**, dann wird **CaseBody1** ausgeführt
- default:** Wenn **Variable** keiner der angegebenen Optionen entspricht, wird **DefaultBody** aufgerufen; optional aber dringend empfohlen
- break:** beendet den **CaseBody**
Wenn das **break** weggelassen wird (*was erlaubt ist*), werden alle nachfolgenden **CaseBodys** ebenfalls ausgeführt – bis zu einem **break**

Syntax:

```
switch (Variable) {  
    case Option1: CaseBody1; break;  
    case Option2: CaseBody2; break;  
    case Option3: CaseBody3; break;  
    default: DefaultBody; break;  
}
```

```
switch (selectedMenuItem) {  
    case 0: say("Back to Menu"); break;  
    case 1: say("Read Messages"); break;  
    case 2: say("Setting"); break;  
    default: say("Invalid"); break;  
}
```

Verschachtelungen

Es ist möglich, innerhalb eines `LoopBody` weitere Schleifen oder Verzweigungen zu programmieren (**nested loops**). Man sollte es aber mit der Verschachtelung nicht übertreiben, denn ansonsten ist der Code nur schwer zu verstehen:

```
for (int zeile = 1; zeile <= 10; zeile++) {  
    for (int spalte = 1; spalte <= 10; spalte++) {  
        if (zeile == spalte) {  
            turnLeft();  
        }  
    }  
}
```

Mittels des Keywords `return` kann der gesamte Ablauf beendet werden. Wird `return` in Karels `run()` Methode verwendet, so wird an dieser Stelle das gesamte Programm beendet.

In manchen Fällen ist es hilfreich, eine Endlosschleife zu programmieren, welche dann innerhalb des `LoopBody` mittels der Keywords `break` oder `return` beendet wird.

```
while (true) {  
    move();  
    if (frontIsBlocked()) {  
        break;  
    }  
}
```

```
public void run() {  
    while (true) {  
        while (frontIsClear()) {  
            if (beeperIsPresent()) {  
                return;  
            }  
        }  
    }  
}
```

Karels Kommandos Und Sensoren

Karels Kommandos

`move()` → gehe einen Schritt vorwärts
`turnLeft()` → Drehung links um 90°
`pickBeeper()` → hebe einen Beeper an aktueller Position auf
`putBeeper()` → lege einen Beeper an aktueller Position ab

Karels Sensoren

- `beeperIsPresent()` → true, falls an aktueller Position ein Beeper liegt (sonst false)
- `frontIsClear()`, `frontIsBlocked()` → Prüfung ob Weg geradeaus frei/blockiert ist
- `leftIsClear()`, `leftIsBlocked()` → Prüfung ob die linke Seite frei/blockiert ist
- `rightIsClear()`, `rightIsBlocked()` → Prüfung ob die rechte Seite frei/blockiert ist
- `facingNorth()`, `facingEast()`,
 `facingSouth()`, `facingWest()` → Prüfung ob Karel nach Norden/Osten/Süden/Westen blickt

Ausgabe von beliebigem Text auf der Konsole

(Nicht spezifisch für Karel)

`System.out.println("Hebe einen Beeper auf.")` → Informative Konsolenausgabe
`System.err.println("Argh, es ist ein Fehler passiert!")` → Fehlermeldung auf der Konsole

Variablen Und Datentypen

Variablen und Konstanten

Variablen

Wenn sich der Computer zur Laufzeit des Programms Informationen eine Zeit lang merken soll, kommen **Variablen** bzw. **Konstanten** zum Einsatz.

Variablen und Konstanten:

- sind Behälter für Werte, die während des Programmablaufs auftreten.
- müssen deklariert werden, bevor sie verwendet werden können:

Syntax:

`type name;`

```
int numberOfBeepers;  
double fuelPrice;  
String firstName;
```

- muss ein Wert zugewiesen (geschrieben) werden, bevor sie das erste Mal gelesen werden

- Variante 1: Deklaration und Zuweisung in zwei Schritten:

Syntax:

`type name;
name = value;`

```
int numberOfBeepers;  
numberOfBeepers = 1;
```

- Variante 2: Deklaration und Zuweisung in einem Schritt:

Syntax:

`type name = value;`

```
int numberOfBeepers = 1;  
double fuelPrice = 1.65;  
String firstName = "Jane";
```

- Variablen kann jederzeit auch ein neuer Wert zugewiesen werden, z.B.:

```
int numberOfBeepers;  
numberOfBeepers = 1;  
numberOfBeepers = 2;
```

```
int numberOfBeepers = 1;  
numberOfBeepers = 2;
```

- dürfen für ihren **Gültigkeitsbereich** nur ein mal deklariert werden:

```
int numberOfBeepers;  
int numberOfBeepers;
```

Variable 'numberOfBeepers' is already defined in the scope

In IntelliJ werden Variablen, denen mehrere Werte zugewiesen werden, immer unterstrichen dargestellt!

Konstanten

Konstanten werden mit dem Keyword **final** deklariert:

```
final int MAXIMUM_SIZE = 1024;
```

Konstanten kann nur ein Mal ein Wert zugewiesen werden:

```
final int MAXIMUM_SIZE = 1024;  
MAXIMUM_SIZE = 2048;  
Cannot assign a value to final variable 'MAXIMUM_SIZE'
```

Namenskonvention

Der Name von Variablen wird in **lowerCamelCase** geschrieben:

```
int numberOfBeepers;  
double fuelPrice;  
String firstName;
```

Der Name von Konstanten wird in **UPPERCASE** geschrieben:

```
final int MAXIMUM_SIZE;  
final String ERROR_TEXT_INVALID_PASSWORD;
```

Reservierte **Java-Keywords** (z.B. **final**, **int**) können nicht als Name verwendet werden, aber dürfen Namensbestandteil sein (z.B. **finalScore**, **internetConnection**).

Alle Namen in Java (nicht nur für Variablen und Konstanten) beachten die Groß- und Kleinschreibung, **firstname** und **firstName** bezeichnen zwei unterschiedliche Variablen.

Die vergebenen Namen sollten sprechend sein, d.h. **numberOfBeepers** ist besser als **nrb**.

Zwar können auch Umlaute und Sonderzeichen im Namen verwendet werden, doch in Hinblick auf internationale Entwicklungs-Teams empfiehlt es sich, Zeichen zu verwenden die auf den meisten Tastaturen zu finden sind. Auch haben es internationale Entwicklungs-Teams leichter, wenn die Variablen in englischer Sprache benannt werden, d.h. **goodsReceiptNumber** ist besser als **warenEingangsNummer**.

Reservierte Java-Keywods

Wie bereits erwähnt, können reservierte Java-Keywods nicht als Namen verwendet werden, diese sind:

abstract	default	goto	package	synchronized
assert	do	if	private	this
boolean	double	implements	protected	throw
break	else	import	public	throws
byte	enum	instanceof	requires	transient
case	exports	int	return	try
catch	extends	interface	short	var
char	final	long	static	void
class	finally	module	strictfp	volatile
const	float	native	super	while
continue	for	new	switch	

Quelle: [Oracle Java Documentation](#)

Gültigkeitsbereich (Scope) von Variablen

In der Regel wird jede Variable nur in einem bestimmten Bereich der Anwendung benötigt. Jede Variable hat deshalb einen begrenzten **Gültigkeitsbereich**.

```
int numberOfBeepers = 5;
if (numberOfBeepers > 1) {
    numberOfBeepers = 1; // das geht!
}
numberOfBeepers = 5; // das geht auch!
```

Die **Lebensdauer** bezeichnet den Zeitraum, in welchem die Variable verwendet werden kann. Wenn das Programm den Gültigkeitsbereich einer Variable verlässt (d.h. die Lebensdauer zu Ende geht), wird der zugehörige Speicherbereich wieder freigegeben. Der **Garbage Collector** kann den Speicherbereich dann aufräumen.

```
int numberOfBeepers = 5;
if (numberOfBeepers > 1) {
    int numberOfCounts = 1;
}
// das geht nicht, weil numberOfCounts
// nur innerhalb des if-Blocks gültig ist
numberOfCounts = 5;
```

Cannot resolve symbol 'numberOfCounts'

Datentypen

Alle Variablen in Java müssen einen Datentyp besitzen, der angibt wie genau die gespeicherten Daten interpretiert werden müssen. Dabei unterscheidet man grundsätzlich zwischen **primitiven Datentypen** und **Referenztypen**.

Primitive Datentypen

Java stellt eine Reihe verschiedener primitiver Datentypen bereit:

Kategorie	Datentyp	Größe	Wertebereich	Bemerkung
Ganzzahlen	<code>int</code>	32 Bit	−2.147.483.648 bis 2.147.483.647 (2,1 Milliarden)	
	<code>long</code>	64 Bit	−9.233.372.036.854.775.808 bis 9.233.372.036.854.775.807 (9,2 Quintillionen)	
	<code>short</code>	16 Bit	−32.768 bis 32.767	Nur empfehlenswert wenn riesige Mengen dieser Zahlen gespeichert werden müssen und der Arbeitsspeicher knapp ist.
	<code>byte</code>	8 Bit	−128 bis 127	
Dezimal-zahlen	<code>double</code>	64 Bit	$\pm 4.39 \cdot 10^{-322}$ bis $\pm 1.7976931348623157 \cdot 10^{308}$	Auch bei normalgroßen Zahlen viel genauer als <code>float</code> und diesem daher vorzuziehen.
	<code>float</code>	32 Bit	$\pm 1.4 \cdot 10^{-45}$ bis $\pm 3.4028235 \cdot 10^{38}$	Generell ziemlich ungenau. Daher besser gleich <code>double</code> verwenden.
Wahrheits-werte	<code>boolean</code>	32 Bit	Die Werte <code>true</code> und <code>false</code> .	Bei <code>boolean</code> -Arrays belegt jeder Wert nur 1 Bit an Arbeitsspeicher.
Einzel-Buchstaben	<code>char</code>	16 Bit	Ein einzelnes von 65.535 möglichen Unicode-Zeichen	

Die primitiven Ganzzahl-Datentypen von Java sind standardmäßig **signed**, d.h. **vorzeichenbehaftet**.

Auch wenn `double` wesentlich genauer ist als `float`, sollte keiner der beiden Datentypen in Bereichen eingesetzt werden, in denen es auf absolute Genauigkeit ankommt (z.B. Finanzen). Hierfür bietet Java die extrem genauen Datentypen `BigDecimal` und `BigInteger` an.

Arithmetik mit primitiven Datentypen

Primitive Datentypen unterstützen eine Reihe von arithmetischen Operatoren:

```
int zwei = 2; // Zuweisung
int drei = zwei + 1; // Addition
int eins = zwei - 1; // Subtraktion
int vier = zwei * 2; // Multiplikation
int sechs = 12 / zwei; // Division
int neun = 79 % 10; // Modulo (Rest nach Ganzzahlteilung)

sechs++; // increment (+1)
sechs--; // decrement (-1)

// Klammersetzung
int acht = -vier + ((zwei * drei) * (3 - 1));
```

Zwar sind Strings streng genommen keine primitiven Datentypen, doch da sie so häufig verwendet werden, gilt der Additions-Operator auch für sie:

```
String username = "Karel";
String greeting = "Hello " + username; // "Hello Karel"
greeting += "!"; // "Hello Karel!"
```

Hinweis:

Strings an sich sind **immutable**, d.h. nicht veränderbar. Bei den gezeigten Operationen wird jeweils ein komplett neuer String erzeugt und der Variable zugewiesen.

Referenztypen

Neben den primitiven Datentypen ermöglicht Java auch die Verwendung von **Referenztypen** (auch **Objekt-Datentypen** genannt). Sie sind der Inhalt des nachfolgenden Kapitels.

Die **Java Class Library** (JCL) liefert bereits mehrere Tausend verschiedener Objekt-Datentypen mit, z.B.:

```
String username = "Karel"; // Zeichenkette
File audioFile;           // Datei oder Verzeichnis im Dateisystem
List<String> userList;     // geordnete Liste an Werten
Exception databaseError;  // Ein spezifischer Programmfehler
JButton playButton;       // Button in einer grafischen Oberfläche
```

Typumwandlung

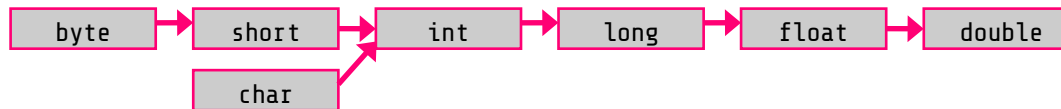
Es kommt häufig vor, dass für ein Programm verschiedene Datentypen gemischt werden. Es ist möglich, innerhalb eines Ausdrucks unterschiedliche Datentypen zu verwenden (z.B. Addition einer Ganzzahl und einer Dezimalzahl). Hierfür müssen die Datentypen jedoch umgewandelt werden. Dabei unterscheidet man zwischen **automatischer Typumwandlung** und **expliziter Typumwandlung**, auch **Casting** genannt.

Automatische Typumwandlung

Bei Bedarf werden kleinere Datentypen automatisch in größere Datentypen umgewandelt:

```
int zwei = 2;
double eineinhalb = 1.5;
double dreieinhalb = eineinhalb + zwei; // geht!
```

Dies funktioniert aber nur gemäß der folgenden Hierarchie:



d.h. das ist im Rahmen der automatischen Typumwandlung nicht erlaubt:

```
int zwei = 2;
double eineinhalb = 1.5;
int dreieinhalb = eineinhalb + zwei; // geht nicht
```

Required type: int
Provided: double

Casting

Hierdurch können größere Datentypen in kleinere Datentypen umgewandelt werden, was aber potentiell mit dem Verlust von Informationen einher geht:

```
int zwei = 2;
double eineinhalb = 1.5;
int drei = (int) eineinhalb + zwei; // geht, aber der Dezimal-Teil ist weg!
```

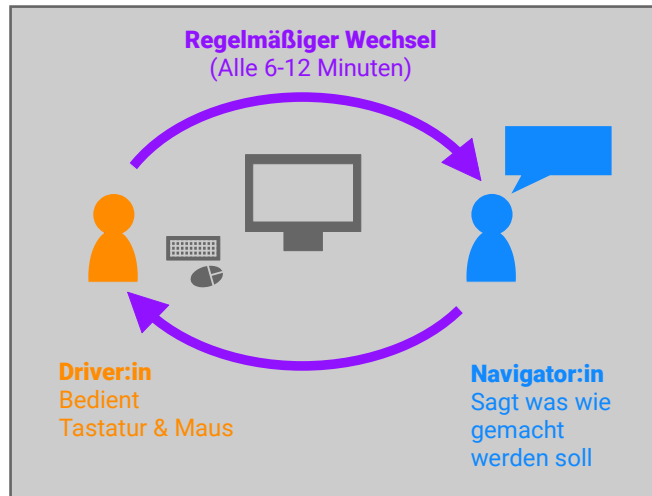
Es können niemals primitive Datentypen in Referenztypen umgewandelt werden oder andersherum. In bestimmten Fällen können allerdings Referenztypen untereinander umgewandelt werden (dazu später mehr).

Pair Programming

Das Erstellen von Software ist ein kreativer Prozess, bei dem häufig Lösungen für komplexe und neue Probleme gefunden werden müssen. Das kann selbst sehr erfahrenen Profis Schwierigkeiten bereiten, was dann wiederum zu Fehlern im Programm führt.

Ein bewährtes Mittel, um besser und schneller Software entwickeln zu können, ist das sogenannte **Pair Programming**, also das Programmieren als Zweiergruppe. In einem ständigen Austausch der Beteiligten werden Ideen diskutiert, Probleme erkannt und behoben – nicht selten bevor sie überhaupt entstehen.

Eine verbreitete Konstellation für eine Pair Programming Session sieht folgendermaßen aus:



Obwohl Pair Programming ursprünglich als lokales Kollaborations-Werkzeug konzipiert worden ist (zwei Personen am selben Computer), eignet es sich ebenso gut auch für die digitale Zusammenarbeit über Video-Anrufe.

Kapitel 1 - Zusammenfassung

- Die **objektorientierte, kompilierte**, General-Purpose-Language Java hat sich in knapp 20 Jahren als eine der populärsten Programmiersprachen überhaupt etabliert
- Der menschen-lesbare Quellcode wird durch den **Compiler** in Java-Bytecode umgewandelt. Dieser wird von der **Java Virtual Machine (JVM)** ausgeführt
- Bei der Entwicklung von Java-Programmen wird das Java Development Kit (JDK) und oft auch eine Integrated Development Environment (IDE) eingesetzt. Für die Ausführung von Java-Programmen reicht oft eine Java Runtime Environment (JRE) aus
- Mit **Schleifen** (**for/while/do**) können Anweisungen mehrfach ausgeführt werden
- Mit **Bedingungen/Verzweigungen** (**if/else/switch**) können in Abhängigkeit eines **Booleschen Ausdrucks** unterschiedliche Anweisungen ausgeführt werden
- **Kommentare** dienen der Erläuterung und Dokumentation des Quellcodes
- **Variablen** und **Konstanten** dienen der Zwischenspeicherung des aktuellen Systemzustands. Bei der Deklaration werden Typ und Name der Variable festgelegt. Bei der Zuweisung wird der zugehörige Wert abgespeichert. Variablen sind nur in ihrem Gültigkeitsbereich verfügbar
- Java unterscheidet zwischen **primitiven Datentypen** (z.B. **int, double, boolean**) und **Referenztypen/Objekt-Datentypen** (z.B. **String, File**). Auf primitive Datentypen können eine ganze Reihe an arithmetischen Operationen angewandt werden (z.B. **+, -, *, /**)
- Die Umwandlung von einem Datentypen in einen anderen geschieht in manchen Fällen automatisch, muss manchmal aber auch manuell durchgeführt werden (**Casting**)
- **Pair Programming** ist die Aktivität, bei der man zu zweit Software erstellt